Improving Deep Reinforcement Learning Interpretability & Performance

Aaryan Pawar
University of Washington
1410 NE Campus Parkway, Seattle, WA
aarynpwr@uw.edu

Arjun Tiwari
University of Washington
1410 NE Campus Parkway, Seattle, WA
atiw@cs.washington.edu

Robert Luiten
University of Washington
1410 NE Campus Parkway, Seattle, WA
robertdl@uw.edu

Abstract

Achieving human-level or superhuman performance in complex strategic games like chess has long since been an objective in the field of deep reinforcement learning, and has had several successes with tools such as AlphaZero. However, these reinforcement learning chess models often suffer from a lack of interpretability after training. This paper proposes two new approaches for improving chess reinforcement learning model's interpretability and performance. First, we propose a novel chess reinforcement learning agent, Off The g-AWRL, which utilized a non-temporal version of the generalized attention weighted reinforcement learning algorithm. Secondly, we will apply the concept of attention to deep neural net chess and create a model called ChessNet which allows us to know which parts of the board are most important. Our findings contribute valuable insights into developing more understandable and better player deep learning solutions for challenging sequential decision-making problems and using this model intelligence to train AGI models.

1. Introduction

The remarkable success of deep reinforcement learning (DRL) in mastering complex tasks, ranging from playing Atari games at a superhuman level to achieving top performance in Go and complex video games, has underscored its potential for tackling intricate sequential decision-making problems [6] [5].

However, a major issue these agents often suffer from significant interpretability issues. While many researchers have managed to deliver various state of the art and superhuman results on tasks ranging from Chess to Atari [6] [8], these models operate as a black box after training, and their choices at time can be difficult to explain. This serves as a major detractor for using reinforcement learning based approaches for solving problems.

Our paper takes two approaches to improving reinforcement learning interpretability, evaluating each through the lens of chess. Taking a look at both of these approaches can help us to understand how we can not only improve understanding, but also performance of these agents as well.

1.1. Why Chess?

Chess, with its rich history as an intellectual pursuit and its well-defined rules, serves as an ideal benchmark for evaluating and advancing Deep Reinforcement Learning methodologies for complex strategic reasoning. While groundbreaking work, such as Google Deep Mind's AlphaZero, has demonstrated the capability of DRL to discover novel and highly effective chess strategies, the training process necessitated extensive self-play, making much of it's generated training data a mystery [6]. Furthermore, it has been repeatedly demonstrated that although reinforcement learning chess agents provide valuable insight, algorithmic approaches like Stockfish still reign supreme while also being significantly more transparent about their though processes [4] [7].

1.2. The Path to AGI

Although many research efforts have been deployed towards improving "reasoning" in Large Language Models, there is still significant debate as to whether models can truly reason or are simply extrapolating responses from their training data. Due to recent advances in the field of Artificial Intelligence Agents, we believe that providing an LLM with the appropriate tools can greatly augment its ability to provide intelligent responses. As such, a secondary research goal of this paper is to interface our deep reinforce-

ment model with a local LLM to determine if any improvements in reasoning ability are detectable.

However, as with most deep learning expert systems, the model is not able to explain its decisions. One solution would be use a large language model. Although utilization of a large language model is a worthwhile future goal, due to current limitations of compute, we elected for simpler explanation scheme.

2. Related Work

2.1. Stockfish

The most commonly used chess engine today is Stockfish, which utilizes alpha-beta pruning to search over various possible combinations of moves. The alpha-beta pruning is essentially an improvement over the minimax algorithm, which utilizes a search tree over all possible move combinations to a certain depth. The nodes at the max depth are leaf nodes, which each get assigned a value based on a deterministic evaluation function. The alpha-beta pruning helps reduce the computational time by eliminating paths which don't need to be searched, while still calculating the optimal move based on the evaluations. Stockfish also utilizes an iterative deepening approach, which essentially continuously increases the depth of computation, using shorter-depth results to help inform deeper searches.

Once the algorithm reaches the maximum set depth for a path of moves, the leaf node is evaluated using the static equation, which gives a score based on factors such as king safety, material count, pawn structure and other aspects of the board state depending on position.

One advantage of Stockfish is its efficiency. Due to more advanced pruning techniques and highly optimized C++ code, it is one of the fastest and highly used engines. Additionally, Stockfish is relatively explainable due to its calculative nature and evaluation function, which includes weighted factors based on different details of the position. So its is much easier for Stockfish users to understand how it evaluates certain positions. One primary disadvantage is that Stockfish relies solely on calculation and heuristics, meaning it is unable to perfectly replicate or improve upon the intuition and positional understanding that humans have to develop long-term strategies [7].

2.2. AlphaZero

The current state-of-the-art in deep neural net approaches to chess is AlphaZero, developed by Google's Deep Mind lab. AlphaZero took a novel self play, reinforcement based approach for learning chess at a superhuman level, along with other two player abstract strategy games such as Shogi and Go [6].

AlphaZero uses a neural network with parameters θ, which outputs a policy vector p which gives the likelihood of each possible upcoming move, and a value vector v which gives the value of each move based off of the current state S of the board:

$$f_\theta(S) = (p, v) \quad \text{where} \quad \begin{cases} p = \text{policy_head_CNN}(S) \\ v = \text{value_head_CNN}(S) \end{cases}$$

After finding the policy and value vectors, AlphaZero utilizes a probabilistic version of Monte Carlo Tree Search. To decide which move to explore during search, AlphaZero uses a score $U(s, a)$ that balances how good the move looks so far, and how promising the network thinks it is. Starting at state s, AlphaZero recursively chooses action $a = argmax_a U(s, a)$ until reaching a leaf node (a state not yet fully expanded). $U(s, a)$ is defined as the following:

$$U(s, a) = Q(s, a) + c_{puct} \cdot P(s, a) \cdot \frac{\sqrt{\sum_b N(s, b)}}{1 + N(s, a)}$$

Where $Q(s, a)$ is the current reward value for that move, $N(s, a)$ represents the visit count for the specified state action pair, c_{puct} controls exploration, and $P(s, a)$ represents the prior probability from the neural network.

After this, AlphaZero uses the state visits to form π, a vector in which each element represents the probability of AlphaZero taking an action a from the state s:

$$\pi(a \mid s) = \frac{N(s, a)^{1/\tau}}{\sum N(s, b)^{1/\tau}}$$

This search returns a vector π that represents a probability distribution over moves, either proportionally or greedily with respect to that state, and where τ controls exploration where a higher value encourages more exploration.

Once AlphaZero reaches a leaf node, it evaluates the state using the neural network:

$$(P(s'), v(s')) = f_\theta(s')$$

AlphaZero runs this around 800 times to create it's policy. This allows AlphaZero to learn over time exclusively through self play as opposed to having to have an outside source, such as another chess bot or training data, to teach it how to play chess. During self play, AlphaZero simply plays against itself and forms a list of the states, π, and final game outcome z (-1 for a loss, 0 for a draw, and 1 for a win).

To find it's parameters, AlphaZero aims to minimize the following loss function, where c represents the regularization term:

$$l = (z - v)^2 - \pi^T log(p) + c||\theta||^2 \text{ [4] [6]}$$

AlphaZero was generalizable for any two player, turn based game and was able to achieve superhuman performance in Chess, Go, and Shogi.

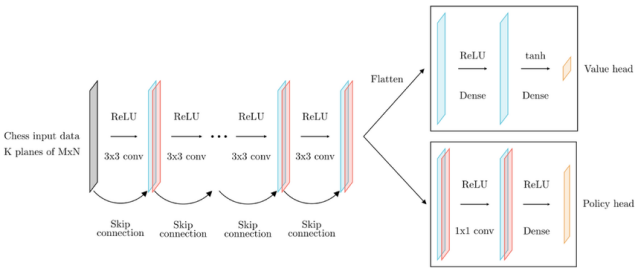

Figure 1. AlphaZero model architecture

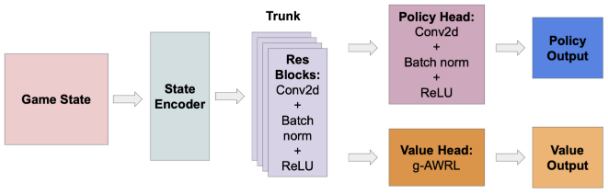

Figure 2. Off The g-AWRLs Architecture

However, this approach was also computationally expensive. The Deep Mind team trained AlphaZero using five thousand first-generation Tensor Processing Units to generate games and 64 Tensor Processing Units to train the neural networks. Training took several days, 41 TPU years, and cost $3e22$ FLOPs. This cost in the neighborhood of thirty thousand to thirty-five million dollars for TPU compute alone. In addition, as with most reinforcement learning algorithms, it was a mystery why AlphaZero made certain decisions based on certain board states [6]. With these barriers of entry, reinforcement chess algorithms haven't been commonly used for learning chess.

2.3. Generalized Attention-Weighted Reinforcement Learning (g-AWRL)

There have been several attempts to implement attention into reinforcement learning algorithms, as it can provide not only better results, but also a better understanding of what relationships between features the model is focusing on [5] [1]. Generalized attention-weighted reinforcement learning (g-AWRL) is a form of weighted reinforcement learning that incorporates attention. It's value function v can be defined as the following:

$$v_t(S) = \sum_{i=1}^{L} W_t^i (\sum_{j=1}^{L} q_t(i) k_t(i)^T e_t(S_j, :))$$

Where S represents the state at time t, $q_t(i)$ represents the query vector at time t for the ith feature, $k_t(j)$ represents the key vector at time t for the jth feature, $e_t(S_j, :)$ represents the embedding for the jth feature in the state S and time t, and W_t^i represents the weight transformation associated with the ith feature at time t.

For each pair of positions, g-AWRL computes the attention scores between feature i and j, multiplying it by it's weight embedding. It is then passed through the weight transformation associated with the values.

g-AWRL bears striking resemblance to classic attention mechanisms. As opposed to traditional reinforcement learning methods, especially with CNNs, g-AWRL is able to find relationships between different elements of the state space. Compared to a traditional weighed reinforcement learning approach, features are not weighed by their individual components, but rather how they interact with other features in the state. In the original paper, g-AWRL Reinforcement learning agents were able to outperform CNN and a-CNN based models in vintage Atari games such as Pong, Demon Attack, and Asteroids, achieving impressive performance boosts and much quicker convergence than all three [1].

3. Methodology

3.1. Off The g-AWRL Chess

We propose a new model, Off The g-AWRL. The architecture is heavily inspired by AlphaZeros, however it utilizes g-AWRL implemented for the value head as opposed to AlphaZero's approach of utilizing a traditional CNN. We use a non-temporal version of g-AWRL, defined as the following:

$$v(S) = \sum_{i=1}^{L} W^i (\sum_{j=1}^{L} q(i) k(i)^T e(S_j, :))$$

After this, as opposed to AlphaZero's f_θ, we define Off The g-AWRLs f_θ as the following:

$$f_\theta(S) = (p, v) \quad \text{where} \quad \begin{cases} p = \text{policy_head_CNN}(S) \\ v = \text{g-AWRL_value_head}(S) \end{cases}$$

This implementation means the network can have all of the benefits associated with g-AWRL when finding the value vector, while also allowing it to use the same loss function as AlphaZero.

Although g-AWRL was primarily developed for use in dynamic, temporal environments, we hope to find that it will generalize well to changes in a chess environment. This implementation should maintain the same benefits as AlphaZero (learning through self play, generalizable), while the addition of g-AWRL should lead to improvements in convergence, and therefore training efficiency, in addition to attention allowing for better interpretability by allowing us to have access to the attention map which the model utilizes.

We will evaluate Off The g-AWRL against an AlphaZero clone at Hexapawn, a simplified version of chess, where we

will compare loss values, win rates, and visualize Off The g-AWRL's attention weights to get a better idea of what the model is focusing on.

3.2. ChessNet: A Convolutional Attention Model Training Scheme

If we look at the previous use of g-AWRL to simpler chess problems (eg. centipawn or hexapawn), we notice that we have cross-attention across every single square/piece on the board. While we demonstrate that this scheme does allow the model to understand the position well (see results), it is also true that this attention implementation is highly inefficient. This is because this square-level attention operation grows as $O(n^2)$ as a function of the size of the board. For the real game of chess, a single attention calculation would involve 64^2 operations. For tasks such as those in Python gym, this operation would involve as much as $(32 \cdot 32 \cdot 3)^2$ operations (for a 32x32 RGB image). As such, we are in need for a new attention implementation.

Luckily, an analysis of the AlphaZero model gives us a solution. Looking at the model architecture below, we see that the architecture relies on deeper and deeper feature maps to get the appropriate representations.

To implement board-level attention, we propose ChessNet, which involves simply adding a convolution attention head at each level which outputs a weight (from 0 to 1) for each square. This allows the model to focus on different areas of the board when feeding inputs into convolution operations by scaling up or down certain features.

4. Results

4.1. Off The g-AWRL Experimentation

As a baseline model for comparison, we implemented a clone of AlphaZero which utilized two residual blocks with 64 hidden convolution layers. We then utilized a convolution layer for both the policy and value head of the model. We then implemented an Off The g-AWRL model with the same number of residual blocks and hidden layers, but replaced the value head of the model with an implementation of non-temporal g-AWRL.

For evaluating the performance and speed of our Off the g-AWRL model, we choose to conduct a comparative study. Such a comparison between our model and AlphaZero will provide insight as to whether a chess model that has explainability as its primary advantage still performs well.

To evaluate our model strength, we simulate 100 games each between our model and the AlphaZero clone above (50 for each of the colors white/black). We perform this test for 3 different versions of Hexapawn (3x3, 6x6, and 9x9). This will allow us to assess how our model performs at a range of complexities instead of simply Hexapawn.

In addition to performance, our objective from this methodology was to assess whether our model, which prioritizes explainability, can still be competitive with state-of-the-art chess engines like AlphaZero in terms of performance and speed. For different versions of Hexapawn with increasing complexity (3x3, 6x6, 9x9,..., 18x18), we simulate 500 games each between Off the g-AWRL and AlphaZero (250 games for each of the colors white/black) to collect compute performance metric data for move time. We then process the data to determine each models average move time for each of the different Hexapawn variants. Having the different Hexapawn variants allows us to evaluate compute efficiency as well as scalability.

4.1.1 g-AWRL Performance

To evaluate performance, we compared the two model's results on Hexapawn, a scaled down version of chess wherein players aim to either stalemate the game, take all of the opponents pawns, or advance one of their pawns to the other side of the board. We trained each model for 100 iterations, wherein the model performed 100 self-play games in each and we kept all hyperparameters equal. For 3x3 hexapawn, each model took approximately 2 minutes to train on an Apple M1 Max processor, with it scaling to 35 minutes for larger boards (i.e. 12 x 12 Hexapawn).

Standard 3x3 Hexapawn is a solved game, so as such whichever player is in the white player position is guaranteed to win the game [2]. As such, our goal for model performance is to have it so that the model can win when in the white position. After training, we found that both the AlphaZero clone and the Off The g-AWRL model converged to optimal play within 10 training iterations, meaning that it showed similar performance. Despite this, the Off The g-AWRL model had significantly more loss, likely due to the value function's scale.

4.1.2 Attention Mechanism Analysis

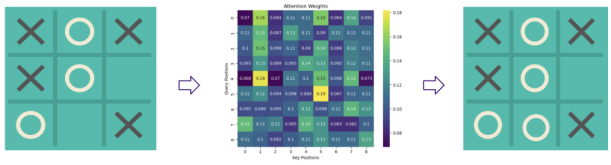

Figure 5. Off The g-AWRL TicTacToe Attention Visualizations

The primary advantage of using Off The g-AWRL is made evident when we visualize the attention weights used for the value head of the model. The visualizations can be found in Figure 4, where each board square is assigned a number 0-8. By visualizing the attention weights, we can find

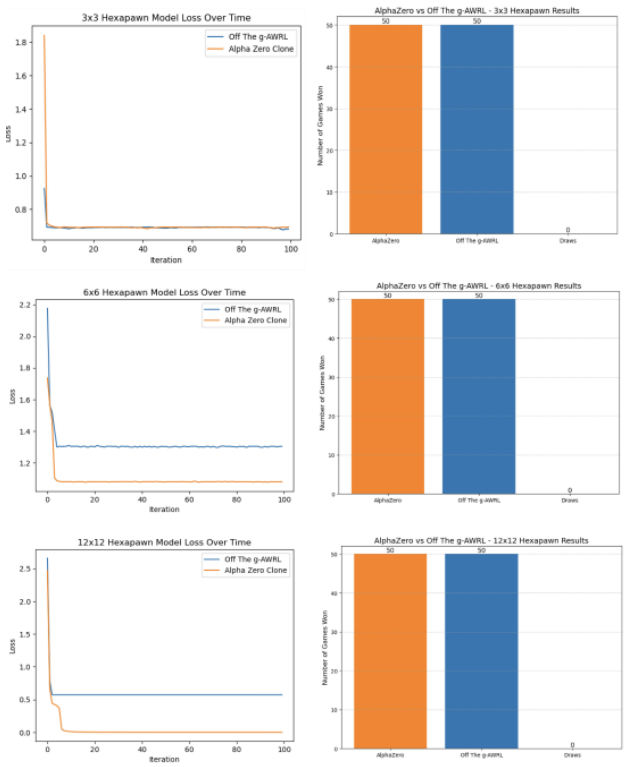

Figure 3. Hexapawn Evaluation Off The g-AWRL vs. AlphaZero clone

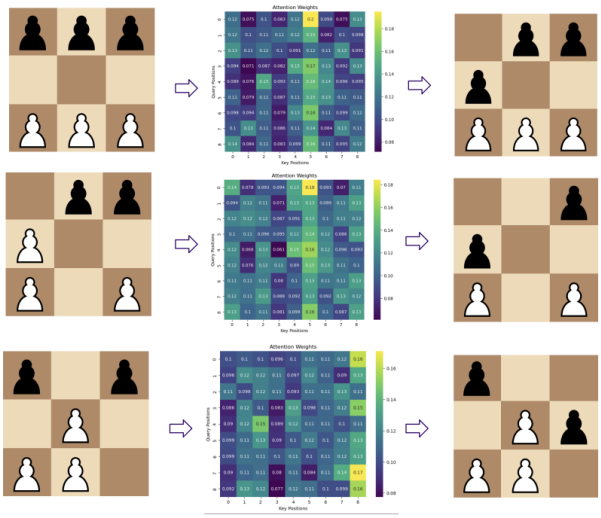

Figure 4. Off The g-AWRL Hexapawn Attention Visualizations

the weights that Off The g-AWRL assigns to different relationships across the board, and can better understand it's thought processes. For example, in the second listed board state, Off The g-AWRL notices that the empty square (in square 0) is about to be occupied by the enemy's white pawn (in square 5), which will make black lose the game. As such, the black pawn quickly takes the white pawn, preventing a loss.

Off The g-AWRL, much like AlphaZero, can generalize for any two player, turn based game as well. We trained an instance of Off The g-AWRL with the same parameters as our Hexapawn implementation on TicTacToe. TicTacToe has a larger state space than Hexapawn, so as such it took 25 iterations to converge to the optimal policy.

Figure 5 shows a visualization of the Off The g-AWRL attention weights for TicTacToe, where each square is similarly marked 0-8. Playing O, we find that Off The g-AWRL senses that the relationships between squares 1 & 4 (where the Os about to form a line are located) is important, so as such it assigns a higher weight to the relationship between those two squares, allowing it to complete the line and win the game.

4.1.3 Computational Efficiency

Efficiency was one of our primary concerns when it came to training our model, as it was expected that adding the attention mechanism would likely increase overhead. Ultimately, we found that the Off The g-AWRL model did experience a slightly longer compute time per move, but it was not a major increase.

Since the attention mechanism calculates relationships between different areas of the input (in this context, the board squares), there are increased operations required for Off the g-AWRL. Despite this, the fact that the increase in compute time (shown in Figure 6) is only slight shows that our implementation in Off the g-AWRL is relatively efficient. It also suggests that the majority of the compute cost still lies in the the MCTS simulations and the numerous forward propagation through the neural network.

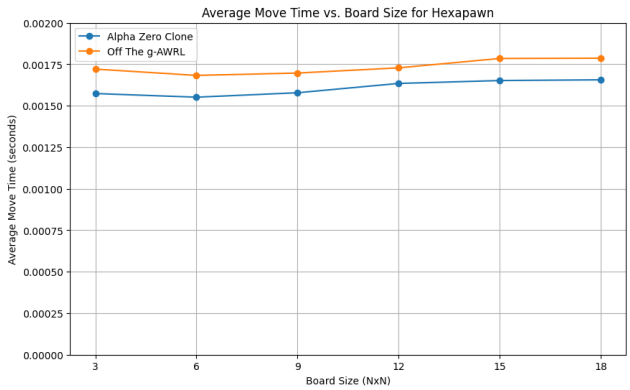

Figure 6. AlphaZero vs. Off The g-AWRL Avg. Move Time (Seconds)

Additionally, it is also clear from Figure 6 that the gap in

	Train Acc.	Test Acc.
Vanilla Convolutional Model	84.05%	90.27%
ChessNet Model	88.34%	94.06%

average move time between Off the g-AWRL and the AlphaZero clone doesn't fluctuate much even as the complexity of the game increases. This aligns with the expected nature of most convolutional layers, which have an inherent property of being proficient at scaling as input size increases. Nonetheless, such a finding demonstrates the scalability of our model, which is crucial when exploring more complex games such as full-scale chess.

4.2. ChessNet's Results

Earlier in this investigation, we contemplated using Monte Carlo Tree Search to generate data to train our model. However, since we were more focused on **efficiency** rather than **performance**, we opted to use chess games between players of ELO values 2000 or higher [3]. Thus, we trained our model to predict the next move given a board on a dataset of fifteen thousand such games for fifty epochs.

A critical eye will see some clear problems in this approach. For one, the games provided in the dataset where from players of a variety of skill levels ranging approximately from an elo of 1500 to 2800. As such, there is no guarantee that the model was learning the best move in a given situation. However, since our goal was to teach the model about the typical habits of human masters and the rules surrounding chess, we judged this step to be adequate. Another reason why this training task appealed to us is that it entirely circumvents the challenge of tree search. Since our goal is to first efficiently explain human chess, we deemed training on human master games was the superior choice. However, a good extension would be to apply model architecture on a AlphaZero clone as well.

After running the vanilla convolutional model and the covolutional attention model we get the following training and test set accuracies.

From this data, it is pretty clear that the addition of attention weighting to features in ChessNet increases classification accuracy by about four percentage points for both the training and test sets. However, this improvement could also be attributed to the fact that ChessNet has more parameters per layer. The only way to resolve this dilemma is to analyze an attention mapping generated by ChessNet. While many of the said mappings were quite obfuscating, we found one such mapping that was quite interesting.

In this board state, we encountered a chess tactic known as a fork. A fork is a tactic where a piece such as a knight attacks two other opposing valuable pieces at the same time (such as a king and a queen). This tactic forces an opponent to save their king at the cost of losing a valuable piece. When we set up a situation like this on a board and gave it to the model, the two pieces to be forked were heavily weighted whereas the rest of the board was almost ignored. While there is still debate as to this behavior generalizes, we believe that further research is necessary for verification.

5. Discussion

5.1. g-AWRL: Pay Attention To The Future

We've shown that g-AWRL can pose significant interpretability advantages when training a model. The observation that our model exhibits competitive performance against AlphaZero while adding only a slightly longer compute time per move is definitely significant. It maintains the generalization of AlphaZero while still scaling relatively well.

The primary challenge which we faced while testing our g-AWRL model was computational power. We did not have the resources needed to train either our AlphaZero clone or Off The g-AWRL on full-sized chess. When experimenting, we found that even a single iteration of training was extremely lengthy due to the complexity of chess (the feature space is much larger). The challenges posed by reinforcement learning algorithms are primarily those of efficiency and interpretability. While Off The g-AWRL resolves the interpretability issue, further improvements are needed to eliminate reinforcement learning's most significant challenge.

5.2. ChessNet Model Analysis

From the given results, it is pretty evident that our model has comparable elo to other previous works. However, through supervised training, we were able to achieve unprecedented model explainability. With a relatively small model, we were able to get reliable short-term analysis of board states done consistently. We believe that this is a step forward in reinforcement learning research. However, we must also note the caveats to our research. First off, there is no real guarantee of alignment between the model's analysis and the ground truth. This means that more validation must be done from human experts to ensure that the model's outputs are truly valid. The logical next step in this analysis would be to use the attention maps generated by ChessNet to create chess captions. This will allow human experts to better verify whether the patterns learned by the model are valid or not.

6. Conclusion

We find that it is possible, and at times beneficial, to implement attention mechanisms such as self-attention and g-AWRL into reinforcement learning models through our implementation of Off The g-AWRL and our token based

chess agent. Not only do these strategies improve interpretability, one of the primary issues associated with classical reinforcement learning, but also these strategies can be shown to achieve near-equal performance in some domains, such as simple games like Hexapawn.

While there is still work to be done in incorporating these strategies into reinforcement learning agents, mainly regarding efficiency as models such as Off The g-AWRL require significant computational power for more complex tasks, we can now understand their though processes, and can ultimately understand more about the models around us.

References

[1] Lennart Bramlage and Aurelio Cortese. Generalized attention-weighted reinforcement learning. *Neural Networks*, 145:10–21, 2022. 3

[2] Martin Gardner. Hexapawn. *Scientific American*, 206(3):118–120, 1962. 4

[3] Mitchell Jordinger. Chess game dataset (lichess) v1, 2017. This is a set of just over 20,000 games collected from a selection of users on the site Lichess.org, and how to collect more. 6

[4] Dominik Klein. Neural networks for chess, 2022. 1, 2

[5] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. Playing atari with deep reinforcement learning, 2013. 1, 3

[6] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dharshan Kumaran, Thore Graepel, Timothy Lillicrap, Karen Simonyan, and Demis Hassabis. Mastering chess and shogi by self-play with a general reinforcement learning algorithm, 2017. 1, 2, 3

[7] The Stockfish developers (see AUTHORS file). Stockfish. 1, 2

[8] David Silver Alex Graves Ioannis Antonoglou Daan Wierstra Volodymyr Mnih, Koray Kavukcuoglu and Martin Riedmiller. Playing atari with deep reinforcement learning, 2013. 1